

Alapfogalmak

- **Sávszélesség (Bandwidth, C):** legnagyobb adatsebesség (b/s). Példa: 1 Gbit/s.
- **Késleltetés (Latency, L):** egy csomag egyirányú ideje (s). **RTT:** oda-vissza idő.
- **Jitter:** késleltetés ingadozása (szórás vagy max-min).
- **Kihasználtság (Utilization, ρ):** terhelés/kapacitás arány.

Prefixek & átváltások

Gyakori átváltások:

- 1 B = 8 b
- 1 MB/s (decimális) = 8 Mb/s
- 1 Gbit/s = 1000 Mbit/s = 10^9 b/s

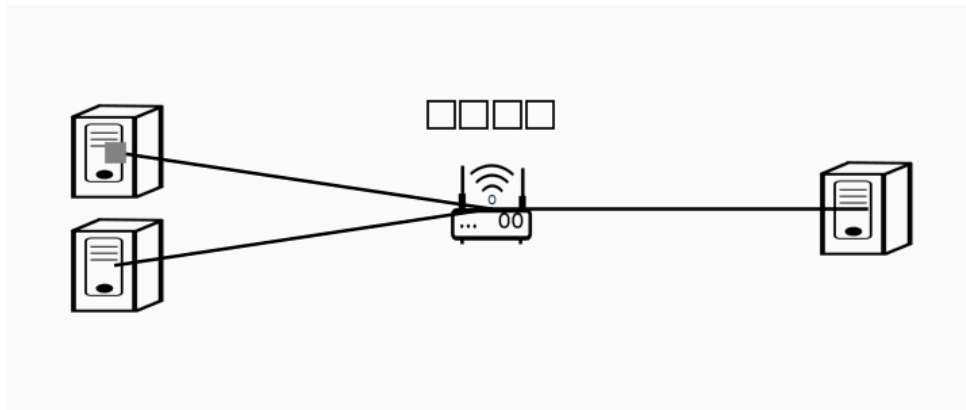
Példa: 125 MB/s = 1000 Mb/s = 1 Gb/s.

Késleltetés

- **Terjedési késleltetés (Propagation, L_p):**
 - Képlet: $L_p = \frac{\text{Távolság}}{\text{terjedési sebesség}}$
 - Vákuum = 3×10^8 m/s; vezeték = 2×10^8 m/s ($\approx 5\mu$ s/km)
- **Egyéb késleltetési összetevők:**
 - L_s : Sorbaállítási késleltetés (Serialization)
 - L_{proc} : Feldolgozási késleltetés (Processing)
 - L_q : Sorbanállási késleltetés (Queuing)
- **RTT képlet:** $\text{RTT} = 2 \cdot L_p + \text{egyéb}$

Gyakorlati példák (egyirányú terjedés):

- 100 km szál: $L_p = 100 \text{ km} \cdot 5\mu \text{ s/km} = 0.5 \text{ ms}$.
- 1000 km szál: $L_p = 1000 \text{ km} \cdot 5\mu \text{ s/km} = 5 \text{ ms}$.



Alap képletek (sebesség, idő, méret)

- **Bitidő:** $t_b = \frac{1}{C}$
- **Idő egy keretre:** $t = \frac{\text{méret_bitben}}{C}$
- **Átvitt adat adott idő alatt:** $\text{adat} = C \cdot t$

Hasznos hatásfok (E):

- $E = \frac{\text{hasznos_biteket}}{\text{összes_átvitt_bit}}$
- $\text{Goodput} = E \cdot C$

Egyszerű feladatok – Kidolgozás és Megoldás

1. Átviteli idő

Feladat: Mennyi idő alatt továbbítunk 60 MB adatot egy 100 Mbit/s linken (overhead nélkül)?

Megoldás:

- Adatméret bitben: $D = 60 \text{ MB} = 60 \cdot 8 \text{ Mbit} = 480 \text{ Mbit} = 480 \cdot 10^6 \text{ bit}$.
- Link sebessége: $C = 100 \text{ Mbit/s} = 100 \cdot 10^6 \text{ bit/s}$.
- Átviteli idő: $t = \frac{D}{C} = \frac{480 \text{ Mbit}}{100 \text{ Mbit/s}} = \mathbf{4.8 \text{ ms}}$

2. Propagációs késés

Feladat: Mennyi a terjedési késleltetés 200 km hosszú optikai szálon ($5\mu \text{ s/km}$)?

Megoldás:

- $L_p = 200 \text{ km} \cdot 5\mu \text{ s/km} = 1000\mu \text{ s} = \mathbf{1 \text{ ms}}$

3. RTT becslés

Feladat: Egy adatsomag 1000 km szálon halad ($5\mu \text{ s/km}$). Becsöld meg az RTT-t, úgy hogy a hálózati eszközök feldolgozása további fix 2 ms késleltetést ad irányonként!

Megoldás:

- Egyirányú terjedési késleltetés: $L_p = 1000 \text{ km} \cdot 5\mu \text{ s/km} = 5 \text{ ms}$.
- Egyirányú teljes késleltetés: $L_{\text{egyirányú}} = L_p + L_{\text{proc}} = 5 \text{ ms} + 2 \text{ ms} = 7 \text{ ms}$.
- Oda-vissza idő (RTT): $\text{RTT} = 2 \cdot L_{\text{egyirányú}} = 2 \cdot 7 \text{ ms} = \mathbf{14 \text{ ms}}$

4. Hasznos hatásfok

Feladat: Maximális csomagméret (MTU) = 1500 B, ebből az IP fejléc (min): 20 B, TCP fejléc (min): 20 B. Számoljuk ki a hasznos hatásfokot százalékban. Mennyi a goodput 1 Gbps linken?

Megoldás:

- Fejlécek összege: $20 \text{ B} + 20 \text{ B} = 40 \text{ B}$.
- Hasznos teher (payload): $1500 \text{ B} - 40 \text{ B} = 1460 \text{ B}$.
- Hasznos hatásfok: $E = \frac{1460}{1500} = \frac{73}{75} \approx \mathbf{97.33\%}$.
- Goodput 1 Gbps linken: $\text{Goodput} = E \cdot C = 0.97333 \cdot 1 \text{ Gbps} = \mathbf{973.33 \text{ Mbps}}$.

Komplexebb feladatok – Kidolgozás és Megoldás

Adatközpont költöztetés – hálózati mentés vs. fizikai szállítás

Adatok: V (TB) adat, 3000 km távolság.

- Hálózati költség: $C_{\text{net}}(V) = 500 + 20V$ [€]
- Fizikai költség: $C_{\text{phys}}(V) = 2000 + 5V$ [€]

a) Költségek 40 TB és 300 TB esetén:

- **40 TB esetén:**
 - ▶ $C_{\text{net}}(40) = 500 + 20 \cdot 40 = 500 + 800 = \mathbf{1300 \text{ €}}$
 - ▶ $C_{\text{phys}}(40) = 2000 + 5 \cdot 40 = 2000 + 200 = \mathbf{2200 \text{ €}}$
 - ▶ **Eredmény:** 40 TB-nál a **hálózati átvitel olcsóbb** ($1300 \text{ €} < 2200 \text{ €}$).
- **300 TB esetén:**
 - ▶ $C_{\text{net}}(300) = 500 + 20 \cdot 300 = 500 + 6000 = \mathbf{6500 \text{ €}}$
 - ▶ $C_{\text{phys}}(300) = 2000 + 5 \cdot 300 = 2000 + 1500 = \mathbf{3500 \text{ €}}$
 - ▶ **Eredmény:** 300 TB-nál a **fizikai szállítás olcsóbb** ($3500 \text{ €} < 6500 \text{ €}$).

b) Egyensúlyi adatméret (V^*):

- Feltétel: $C_{\text{net}}(V^*) = C_{\text{phys}}(V^*)$

- $500 + 20V^* = 2000 + 5V^* \rightarrow 15V^* = 1500 \rightarrow V^* = \mathbf{100\ TB}$
- $V < 100\ TB$ esetén a hálózat, $V > 100\ TB$ esetén a fizikai szállítás éri meg jobban.

c) Átviteli idő 10 Gbit/s linken (80% hatékonyság) 100 TB-ra:

- Hatékony sávszélesség: $C_{\text{eff}} = 0.8 \cdot 10\ \text{Gbit/s} = 8\ \text{Gbit/s} = 8 \cdot 10^9\ \text{bit/s}$.
- Adatméret (decimális konvencióval): $100\ \text{TB} = 100 \cdot 10^{12}\ \text{B} = 800 \cdot 10^{12}\ \text{bit}$.
- Idő: $t = \frac{800 \cdot 10^{12}\ \text{bit}}{8 \cdot 10^9\ \text{bit/s}} = 10^5\ \text{s} = \mathbf{27.78\ \text{óra}}$ (kb. 1 nap 3 óra 47 perc).
- **(Bináris 1 TiB = $2^{40}\ \text{B}$ esetén: $t \approx 30.54\ \text{óra}$.)**
- **Összehasonlítás:** A hálózati átvitel (27.8 óra) lényegesen gyorsabb a fizikai szállítás **3 napos (72 óras)** átfutási idejénél, miközben 100 TB-nál a költségük azonos (2500 €).

Hibrid adatmentési stratégia

Költségmodell: $C_{\text{hibrid}}(N) = F + a \cdot N + b \cdot (V - N) + c \cdot (V - N)^2$, ahol $0 \leq N \leq V$.

1. Első derivált és lokális szélsőérték (N^*):

- Deriváljunk N szerint (a láncszabályt alkalmazva a $(V - N)^2$ tagra):

$$\frac{dC_{\text{hibrid}}}{dN} = a - b + c \cdot 2(V - N) \cdot (-1) = a - b - 2c(V - N)$$

- Szélsőérték feltétele ($\frac{dC}{dN} = 0$):

$$a - b - 2c(V - N^*) = 0 \rightarrow 2c(V - N^*) = a - b \rightarrow V - N^* = \frac{a - b}{2c}$$

$$N^* = V - \frac{a - b}{2c}$$

2. Második derivált és a szélsőérték jellege:

- $\frac{d^2C_{\text{hibrid}}}{dN^2} = \frac{d}{dN}[a - b - 2cV + 2cN] = 2c$
- Mivel $c > 0$, így a második derivált szigorúan pozitív ($2c > 0$), tehát az N^* pontban **lokális minimum** van.

3. Numerikus példa:

- Paraméterek: $V = 150\ \text{TB}$, $F = 3500\ \text{€}$, $a = 30\ \text{€/TB}$, $b = 20\ \text{€/TB}$, $c = 0.05\ \text{€/TB}^2$.
- **N^* kiszámítása:** $N^* = 150 - \frac{30 - 20}{2 \cdot 0.05} = 150 - \frac{10}{0.1} = 150 - 100 = \mathbf{50\ TB}$
 - **Intervallum ellenőrzése:** $N^* = 50\ \text{TB} \in [0, 150]$, tehát az érvényes tartományba esik.
- **Költségek kiszámítása:**
 - $C(N^*) = C(50) = 3500 + 30(50) + 20(100) + 0.05(100)^2 = 3500 + 1500 + 2000 + 500 = \mathbf{7500\ €}$
 - $C(0) = 3500 + 30(0) + 20(150) + 0.05(150)^2 = 3500 + 3000 + 1125 = \mathbf{7625\ €}$
 - $C(150) = 3500 + 30(150) + 20(0) + 0.05(0)^2 = 3500 + 4500 + 0 = \mathbf{8000\ €}$
- **Magyarázat:** Ha $N < 50$ (túl kevés fizikai szállítás), a hálózati kvadratikusan torlódási díj túlságosan megdrágítja az átvitelt. Ha $N > 50$ (túl sok fizikai szállítás), a fizikai szállítás magasabb alapidóját ($a = 30$ vs $b = 20$) fizetjük feleslegesen. Az $N^* = 50\ \text{TB}$ biztosítja a minimális összköltséget (7500 €).

Keretezés – bit és bájtbeszúrás

Bitbeszúrás (bit stuffing)

- **Használata:** pl. HDLC és CAN protokollokban.
- **Működése:** minden 5 egymást követő 1-es bit után a küldő automatikusan 0-at szúr be.

- **Cél:** biztosítani, hogy a kerethatároló speciális bitmintája (pl. 01111110) ne jelenjen meg a hasznos adatban.
- A vevő oldalon a dekódoló eltávolítja ezeket a beszúrt 0-kat.

Példa:

- Adat: 01111110 (megegyezik a kerethatárt jelző flag-gel)
- Küldő bit stuffing után: 011111010
- Vevő: felismeri és eltávolítja a beszúrt 0-t, így visszakapja az eredeti bitmintát.

Bájtbeszúrás (byte stuffing, karakterbeszúrás)

- **Használata:** pl. PPP és más byte-orientált protokollokban.
- **Működése:** ha a hasznos adat tartalmazza a vezérlő karaktert (pl. 0x7E = keretflag), akkor előtte beszúrnak egy escape karaktert (pl. 0x7D).
- Ha az adat maga escape karaktert (0x7D) tartalmaz, akkor azt is kiegészítik (pl. 0x7D → 0x7D7D).
- A vevő oldalon az escape szekvenciákat visszaalakítják az eredeti adatra.

Példa:

- Keret flag: 0x7E, Escape: 0x7D
- Adat: 0xAA → nincs átalakítás: 0xAA
- Adat: 0x7E → átalakítva: 0x7D7E
- Adat: 0x7D → átalakítva: 0x7D7D

Összehasonlítás

- **Bit stuffing:** bitenként működik, bitorientált protokollokban.
- **Byte stuffing:** bájtanként működik, karakterorientált protokollokban.

Mindkét módszer biztosítja, hogy a vezérlő szimbólumok ne keveredjenek az adattal, de egyben növelik a tényleges adat méretét (overhead).

Feladatok – Kidolgozás és Megoldás

1. Bitbeszúrás feladat

Feladat: Az 1111111001111111 üzenet hogy fog kinézni a bitbeszúrásos módszer (5 egymást követő 1-es után 0 beszúrása) alkalmazása után?

Megoldás:

- Bontsuk elemeire az eredeti bitfolyamot az 1-esek számlálásával:
 - 1 1 1 1 1 (5 db 1-es) → **beszúrás: 0**
 - 1 1 0 0 (folytatás, a számláló törlődik a nulláknál)
 - 1 1 1 1 1 (újabb 5 db 1-es) → **beszúrás: 0**
- Eredeti: 11111 1 100 11111
- Beszúrt bitekkel: 1111101100111110
- **Végeredmény: 1111101100111110**

2. Bájtbeszúrás feladat

Feladat: Hogy kerül átvitelre a következő adat:

A	B	C	FLAG	D	ESC	FLAG	E	F	FLAG	FLAG	G
---	---	---	------	---	-----	------	---	---	------	------	---

Megoldás:

- Minden FLAG elé egy ESC karaktert szúrunk be.
- Minden ESC elé egy ESC karaktert szúrunk be.
- Átalakítás lépésről lépésre:

- FLAG → ESC FLAG
- ESC → ESC ESC
- **Végeredmény (adatmező):** [A] [B] [C] [ESC] [FLAG] [D] [ESC] [ESC] [ESC] [FLAG] [E] [F] [ESC] [FLAG] [ESC] [FLAG] [G]

(A teljes keretnél a keret elejére és végére egy-egy nyitó/záró [FLAG] is kerül.)

3. Kerethossz és eredeti adathossz feladat

Feladat: Bájtt-alapú protokoll, keretek nyitó és záró FLAG bájtal. 8 keret küldése, összhossz a médiumon: 2048 Byte. Az eredeti adatban 16 ESC és 8 FLAG bájt volt. Hány bájtból állt az eredeti adat?

Megoldás:

- Jelölje D_{orig} az eredeti adat bájtszámát.
- **1. Kerethatárolók:** 8 keret $\times$ 2 (FLAG keretenként) = 16 Byte overhead.
- **2. Bájtbeszúrás overhead:**
 - 8 eredeti FLAG miatt +8 ESC bájt beszúrása.
 - 16 eredeti ESC miatt +16 ESC bájt beszúrása.
 - Összes beszúrt bájt (overhead) = 8 + 16 = 24 Byte.
- **Egyenlet:**

$$\text{Teljes méret} = D_{\text{orig}} + \text{Kerethatárolók} + \text{Beszúrt ESC bájtok}$$

$$2048 = D_{\text{orig}} + 16 + 24$$

$$2048 = D_{\text{orig}} + 40$$

$$D_{\text{orig}} = 2048 - 40 = \mathbf{2008 \text{ Byte}}$$